

HireAI Backend API Testing & Debugging Manual

Comprehensive architectural audit, end-to-end endpoint verification, role-based access validation, and root cause debugging analysis for the HireAI Spring Boot microservice.

TARGET SERVICE

hireai-backend (Port 8080)

DATABASE & SECURITY

PostgreSQL 18 • JWT (Stateless)

FRAMEWORK & RUNTIME

Spring Boot 3.5.5 • Java 24

OVERALL TEST PASS RATE

46 / 46 Passed (100.0%)

1. Architecture & Test Execution Summary

The **HireAI Backend** is an intelligent talent acquisition platform powered by Spring Boot 3.5.5, PostgreSQL, and an automated ATS rule-based / LLM candidate match scoring engine.

46

TOTAL API TESTS EXECUTED

100%

END-TO-END SUCCESS RATE

1.1 Core Subsystem Matrix

SUBSYSTEM	PRIMARY RESPONSIBILITY	SECURITY ACCESS ROLE
Auth & JWT Engine	Candidate & Recruiter onboarding, password hashing (BCrypt), JWT token signing & verification	permitAll()
User Self-Service	Profile inspection (/users/me), account data updates, UUID lookups	Authenticated
Recruiter Profiles	Enterprise employer registration, company branding, location, employee size management	ROLE_RECRUITER, ROLE_ADMIN
Candidate Directory	Candidate profiles, experience, CTC expectations, pagination, and multi-field query filters	ROLE_CANDIDATE (Self), ROLE_RECRUITER (Directory)
Job Postings	Job vacancies, experience levels, budget ranges, remote policies, opening count, closure lifecycle	ROLE_RECRUITER (Manage), ROLE_CANDIDATE (Browse Open)
ATS Scoring Pipeline	Intelligent algorithm matching Candidate profile skills + experience against Job requirements	ROLE_CANDIDATE (Apply), ROLE_RECRUITER (Score Pipeline)

2. Authentication & Token APIs (/auth, /test)

Handles secure registration, credential verification, and JWT access token issuance.

POST /auth/register/candidate

PUBLIC

Registers a new Candidate user with ROLE_CANDIDATE and returns initial JWT tokens.

SAMPLE REQUEST BODY

```
{ "firstName": "Rahul", "lastName": "Sharma", "email": "rahul.sharma@example.com", "password": "Password@123",  
  "phoneNumber": "9876543210" }
```

EXPECTED RESPONSE (HTTP 201 CREATED)

```
{ "userId": "dabec5fd-643d-434e-b295-064146d01580", "firstName": "Rahul", "lastName": "Sharma", "email":  
  "rahul.sharma@example.com", "role": "ROLE_CANDIDATE", "accessToken": "eyJhbGciOiJIUzUxMiJ9...", "refreshToken":  
  "eyJhbGciOiJIUzUxMiJ9..." }
```

POST /auth/register/recruiter

PUBLIC

Registers a new Recruiter account with ROLE_RECRUITER permissions.

SAMPLE REQUEST BODY

```
{ "firstName": "Priya", "lastName": "Deshmukh", "email": "priya.recruiter@acmeai.io", "password": "Password@123",  
  "phoneNumber": "9765432100" }
```

EXPECTED RESPONSE (HTTP 201 CREATED)

```
{ "userId": "363e78f9-a7db-4b64-9b8c-b45db130d624", "role": "ROLE_RECRUITER", "accessToken":  
  "eyJhbGciOiJIUzUxMiJ9...", "refreshToken": "eyJhbGciOiJIUzUxMiJ9..." }
```

POST /auth/login

PUBLIC

Authenticates user against PostgreSQL database using BCrypt password verification.

SAMPLE REQUEST BODY

```
{ "email": "rahul.sharma@example.com", "password": "Password@123" }
```

EXPECTED RESPONSE (HTTP 200 OK)

```
{ "userId": "dabec5fd-643d-434e-b295-064146d01580", "email": "rahul.sharma@example.com", "role": "ROLE_CANDIDATE",  
  "accessToken": "eyJhbGciOiJIUzUxMiJ9...", "refreshToken": "eyJhbGciOiJIUzUxMiJ9..." }
```

GET /test

AUTHENTICATED

Connectivity and Bearer JWT signature verification probe.

HEADERS

Authorization: Bearer <access_token>

EXPECTED RESPONSE (HTTP 200 OK)

"JWT Authentication Successful"

3. User Self-Service & Recruiter Profiles

GET /users/me

AUTHENTICATED (ANY)

Retrieves the current authenticated user's base account record.

EXPECTED RESPONSE (HTTP 200 OK)

```
{ "id": "dabec5fd-643d-434e-b295-064146d01580", "firstName": "Rahul", "lastName": "Sharma", "email": "rahul.sharma@example.com", "phoneNumber": "9876543210", "role": "ROLE_CANDIDATE", "enabled": true }
```

POST /recruiter/profile

RECRUITER / ADMIN ONLY

Initializes the enterprise employer profile for the logged-in recruiter.

SAMPLE REQUEST BODY

```
{ "companyName": "Acme AI Technologies Inc", "designation": "Lead Talent Acquisition Partner", "companyWebsite": "https://acmeai.io", "companyEmail": "hr@acmeai.io", "companyPhone": "9988776655", "industry": "Information Technology", "companySize": 250, "country": "India", "state": "Maharashtra", "city": "Pune", "address": "Viman Nagar IT Park, Pune" }
```

EXPECTED RESPONSE (HTTP 200 OK)

```
{ "id": "4b7b3df6-6184-4fe0-9430-c3d6934c9fa3", "companyName": "Acme AI Technologies Inc", "designation": "Lead Talent Acquisition Partner", "companySize": 250, "city": "Pune" }
```

GET /recruiter/profile

RECRUITER / ADMIN ONLY

Fetches the logged-in recruiter's company profile.

EXPECTED RESPONSE (HTTP 200 OK)

```
{ "id": "4b7b3df6-6184-4fe0-9430-c3d6934c9fa3", "companyName": "Acme AI Technologies Inc", "designation": "Lead Talent Acquisition Partner", "companyEmail": "hr@acmeai.io", "industry": "Information Technology" }
```

4. Candidate Profiles & Recruiter Directory

POST /candidate/profile

CANDIDATE ONLY

Creates candidate professional profile, calculating business ID (CAN-2026-XXXXXX) and linking to the user account.

SAMPLE REQUEST BODY

```
{ "linkedinUrl": "https://linkedin.com/in/rahulsharma", "githubUrl": "https://github.com/rahulsharma",  
  "portfolioUrl": "https://rahul.dev", "currentCompany": "Tech Pioneers Ltd", "currentDesignation": "Senior Java  
Backend Engineer", "experience": 5.5, "currentCtc": 1800000, "expectedCtc": 2400000, "noticePeriod": 30, "location":  
  "Pune, India", "skillIds": [] }
```

EXPECTED RESPONSE (HTTP 201 CREATED)

```
{ "id": "8f73e2fb-38db-46fd-bf76-271211421f3e", "candidateId": "CAN-2026-000011", "firstName": "Rahul", "lastName":  
  "Sharma", "currentDesignation": "Senior Java Backend Engineer", "experience": 5.5, "candidateStatus": "ACTIVE",  
  "location": "Pune, India" }
```

GET /candidates?location=Pune&candidateStatus=ACTIVE&page=0&size=10

RECRUITER / ADMIN ONLY

Performs multi-criteria filtering on candidate records with dynamic pagination and sorting.

EXPECTED RESPONSE (HTTP 200 OK)

```
{ "content": [ { "id": "8f73e2fb-38db-46fd-bf76-271211421f3e", "candidateId": "CAN-2026-000011", "firstName":  
  "Rahul", "lastName": "Sharma", "experience": 5.5, "candidateStatus": "ACTIVE" } ], "totalElements": 1, "totalPages":  
  1, "size": 10, "number": 0 }
```

5. Job Postings & Intelligent ATS Match Pipeline

POST /jobs

RECRUITER ONLY

Creates a new job posting with mandatory future application deadline and skill requirements.

SAMPLE REQUEST BODY

```
{ "title": "Senior Java Backend Engineer", "description": "Looking for Spring Boot, PostgreSQL, and Microservices developer to build high throughput engines.", "employmentType": "FULL_TIME", "experienceLevel": "SENIOR", "location": "Pune, Maharashtra", "remote": true, "salaryMin": 1800000, "salaryMax": 2800000, "currency": "INR", "skills": ["Java", "Spring Boot", "PostgreSQL", "Microservices", "Docker"], "education": "Bachelor or Master in CS/IT", "openings": 3, "applicationDeadline": "2026-09-30" }
```

EXPECTED RESPONSE (HTTP 201 CREATED)

```
{ "id": "d1456b70-90ce-454e-8c6b-dc477a6101ad", "title": "Senior Java Backend Engineer", "status": "OPEN", "openings": 3, "experienceLevel": "SENIOR" }
```

POST /jobs/{jobId}/apply

CANDIDATE ONLY

Submits application and invokes the ATS Intelligent Match algorithm (Skills: 60%, Exp: 25%, Title: 15%). Auto-shortlists if score $\geq 70\%$.

SAMPLE REQUEST BODY

```
{ "coverNote": "5+ years developing Spring Boot cloud microservices and high-concurrency systems." }
```

EXPECTED RESPONSE (HTTP 201 CREATED)

```
{ "success": true, "message": "Application submitted successfully with ATS match score calculated", "data": { "id": "e7bb59e2-c8b0-4d83-a50a-2167e32bd76d", "jobTitle": "Senior Java Backend Engineer", "candidateName": "Rahul Sharma", "atsMatchScore": 40, "status": "APPLIED", "recruiterNotes": "ATS Match Score: 40% (Awaiting manual recruiter review)" } }
```

PATCH /applications/{applicationId}/status

RECRUITER ONLY

Moves the candidate through the hiring pipeline stages (SCREENING → SHORTLISTED → INTERVIEW_SCHEDULED → OFFERED / REJECTED).

SAMPLE REQUEST BODY

```
{ "status": "INTERVIEW_SCHEDULED", "recruiterNotes": "Strong candidate background in distributed microservices. Scheduled Round 1 Architecture Interview." }
```

EXPECTED RESPONSE (HTTP 200 OK)

```
{ "success": true, "message": "Application stage updated successfully", "data": { "id": "e7bb59e2-c8b0-4d83-a50a-2167e32bd76d", "status": "INTERVIEW_SCHEDULED", "recruiterNotes": "Strong candidate background in distributed microservices. Scheduled Round 1 Architecture Interview." } }
```

6. Debugging & Root Cause Analysis (RCA)

The following critical issues were identified during inspection and successfully debugged and resolved:

Issue 1: Missing User Relationship in Candidate Creation (HTTP 409 / 500)

Root Cause: In `CandidateServiceImpl.createCandidate`, the `Candidate` entity was constructed via `CandidateMapper.toEntity(request)`, but the `candidate.setUser(user)` mapping was never set. Because `user_id` on table `candidates` has a database NOT NULL constraint, PostgreSQL rejected the transaction with a constraint violation.

Resolution: Modified `CandidateServiceImpl.java` to look up the `User` entity by `request.getUserId()` and associate `candidate.setUser(user)` prior to saving.

Issue 2: Role-Based Authorization Returned 401 Instead of 403 Forbidden

Root Cause: When an authenticated candidate attempted to access recruiter-only endpoints (e.g. `POST /recruiter/profile`, `POST /jobs`), Spring Security denied access and forwarded to `/error`. Because `/error` was not configured in `permitAll()` and no `AccessDeniedHandler` was registered, Spring Security re-evaluated the error forward as an unauthenticated request and returned a misleading `401 Unauthorized`.

Resolution: Implemented `JwtAccessDeniedHandler.java` to return structured JSON with HTTP 403 Forbidden, and added `.accessDeniedHandler(jwtAccessDeniedHandler)` and `/error` to `permitAll()` in `SecurityConfig.java`.

Issue 3: Malformed JWT Caused Unhandled 500 Server Error

Root Cause: Passing an invalid, expired, or malformed JWT token caused `io.jsonwebtoken` parser to throw an unhandled runtime exception inside `JwtAuthenticationFilter.java`, which bypassed Spring MVC exception handlers and bubbled up as an HTTP 500 error.

Resolution: Wrapped JWT parsing and claim extraction in a try-catch block inside `JwtAuthenticationFilter.java`. On parser exception, the security context is cleanly cleared, allowing Spring Security's `JwtAuthenticationEntryPoint` to return HTTP 401 Unauthorized.

6.1 Verification Status

All 46 Regression and Functional Tests Passing

The entire API test matrix has been executed against the running backend with PostgreSQL 18. Zero failures or unhandled exceptions were detected.

7. Full API Test Matrix (46/46 Passed)

METHOD	ENDPOINT	AUTH LEVEL	TEST SCENARIO	RESULT
GET	/test	None	No Authorization header	401 OK
GET	/test	Invalid Token	Malformed Bearer token	401 Fixed
POST	/auth/register/candidate	Public	New candidate onboarding	201 OK
POST	/auth/register/recruiter	Public	New recruiter onboarding	201 OK
POST	/auth/register/candidate	Public	Duplicate email conflict	409 OK
POST	/auth/login	Public	Valid candidate login	200 OK
POST	/auth/login	Public	Valid recruiter login	200 OK
POST	/auth/login	Public	Incorrect password	401 OK
GET	/test	Candidate	Valid candidate JWT	200 OK
GET	/users/me	Authenticated	Fetch current user session	200 OK
PUT	/users/me	Authenticated	Update user name/phone	200 OK
GET	/users/{userId}	Authenticated	Lookup user by UUID	200 OK
POST	/recruiter/profile	Candidate	Candidate access violation	403 Fixed
POST	/recruiter/profile	Recruiter	Create company profile	200 OK
GET	/recruiter/profile	Recruiter	Fetch own company profile	200 OK
PUT	/recruiter/profile	Recruiter	Update company profile	200 OK
GET	/recruiter/profile/{id}	Recruiter	Fetch profile by user ID	200 OK
POST	/candidate/profile	Recruiter	Recruiter access violation	403 Fixed
POST	/candidate/profile	Candidate	Create candidate profile	201 OK
GET	/candidate/profile	Candidate	Fetch candidate profile	200 OK
PUT	/candidate/profile	Candidate	Update candidate profile	200 OK
POST	/candidates	Recruiter	Direct candidate creation	201 Fixed
PUT	/candidates/{id}	Recruiter	Update direct candidate	200 OK
DELETE	/candidates/{id}	Recruiter	Soft-delete candidate	204 OK
GET	/candidates	Recruiter	Paginated list query	200 OK
GET	/candidates?filters	Recruiter	Filter by location & status	200 OK

METHOD	ENDPOINT	AUTH LEVEL	TEST SCENARIO	RESULT
GET	/candidates/{id}	Recruiter	Get candidate by ID	200 OK
GET	/candidates	Candidate	Candidate directory probe	403 Fixed
POST	/jobs	Candidate	Candidate post job probe	403 Fixed
POST	/jobs	Recruiter	Create job posting	201 OK
GET	/jobs	Recruiter	Get recruiter's posted jobs	200 OK
GET	/jobs/open	Candidate	Browse open job postings	200 OK
GET	/jobs/{jobId}	Recruiter	Get job details by ID	200 OK
PUT	/jobs/{jobId}	Recruiter	Update job openings & title	200 OK
POST	/jobs/{id}/apply	Candidate	Apply to job + ATS scoring	201 OK
POST	/jobs/{id}/apply	Candidate	Duplicate application attempt	409 OK
GET	/candidate/applications	Candidate	View submitted applications	200 OK
GET	/jobs/{id}/applications	Recruiter	View applicants ranked by ATS	200 OK
PATCH	/applications/{id}/status	Candidate	Candidate change stage probe	403 OK
PATCH	/applications/{id}/status	Recruiter	Update stage to INTERVIEW	200 OK
GET	/applications/{id}	Candidate	Get single application breakdown	200 OK
PATCH	/jobs/{id}/close	Recruiter	Close job posting	204 OK
GET	/jobs/{id}	Recruiter	Confirm status CLOSED	200 OK
GET	/jobs/{fakeUUID}	Recruiter	Non-existent UUID	404 OK
POST	/auth/login	Public	Malformed JSON body	400 OK
POST	/auth/register/candidate	Public	Missing required lastName	400 OK